

Data Manipulation and Cleaning

Data Manipulation & Cleaning (Pandas Library)

Data manipulation and cleaning are the foundation of every data analysis or machine learning project. Real-world datasets are rarely perfect — they often contain missing values, duplicates, inconsistencies, or irrelevant information.

What is Pandas?

Pandas is a powerful Python library for data manipulation and analysis. It provides two main data structures:

- **Series**: One-dimensional labeled array
- **DataFrame**: Two-dimensional labeled data structure (like a spreadsheet or SQL table)

Concept:

Pandas bridges the gap between raw data and analysis. It allows you to clean, filter, reshape, and analyze datasets efficiently. It is built on **NumPy**, so it's optimized for fast numerical operations.

Key Features

- Handling missing data
- Data alignment and integrated handling
- Reshaping and pivoting datasets
- Label-based slicing, indexing, and subsetting
- Merging and joining datasets
- Time series functionality

Pandas is ideal for structured data (rows and columns). It is extensively used in **data analysis**, **finance**, **research**, and **machine learning** for preprocessing and exploring data. It also includes powerful methods like `groupby`, `merge`, and `pivot_table` for summarizing, combining, and reshaping data.

1. Core Data Structures: Series and DataFrame

Series

A **Series** is a one-dimensional labeled array capable of holding data of any type (integers, floats, strings, etc.). Each element has an **index** (label).

Example:

```
import pandas as pd
# Creating a Series
data = [10, 20, 30, 40]
index_labels = ['a', 'b', 'c', 'd']
series = pd.Series(data, index=index_labels)
```

Here, numbers 10–40 are data values, while 'a'–'d' are their labels. You can access data using labels like `series['a']`.

Output: →

Index	
a	Age
a	b
c	c
d	40
10	40

Use Case:

Used for 1D data — prices, scores, or sensor readings.

DataFrame

A **DataFrame** is a two-dimensional labeled tabular structure with rows and columns. It can be thought of as a collection of Series sharing the same index.

Example:

```
# Creating a DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 28],
    'City': ['New York', 'London', 'Tokyo', 'Paris']
}
df = pd.DataFrame(data)
```

Name	Age	City
Alice	25	New York
Bob	30	London
Charlie	35	Tokyo
David	28	Paris

Each key becomes a column name. Pandas automatically creates indices for rows.

Operations:

- Selecting columns: `df['Name']`
- Filtering: `df[df['Age'] > 30]`
- Adding a column: `df['Country'] = 'USA'`
- Dropping a column: `df.drop('City', axis=1)`

DataFrame: multiple Series aligned by a shared index. It forms the foundation for all Pandas operations.

2. Importing Data and Initial Cleaning

Datasets often come from files like CSV, Excel, or databases.

Example:

```
df = pd.read_csv('data.csv')
```

Once imported, the cleaning process begins.

Common Cleaning Steps:**1. Checking for missing values:**

```
df.isnull().sum()
```

2. Handling missing values:

- Fill missing data:
`df['column'].fillna(value, inplace=True)`
- Drop missing rows or columns:
`df.dropna(inplace=True)`

3. Changing data types:

```
df['Age'] = df['Age'].astype(int)
```

4. Renaming columns:

```
df.rename(columns={'OldName': 'NewName'}, inplace=True)
```

5. Removing duplicates:

```
df.drop_duplicates(inplace=True)
```

These ensure the dataset is in a usable form for further analysis.

Real-world Example:

While cleaning survey data, you might replace blank responses with averages, remove duplicate entries, and standardize text capitalization before analysis.

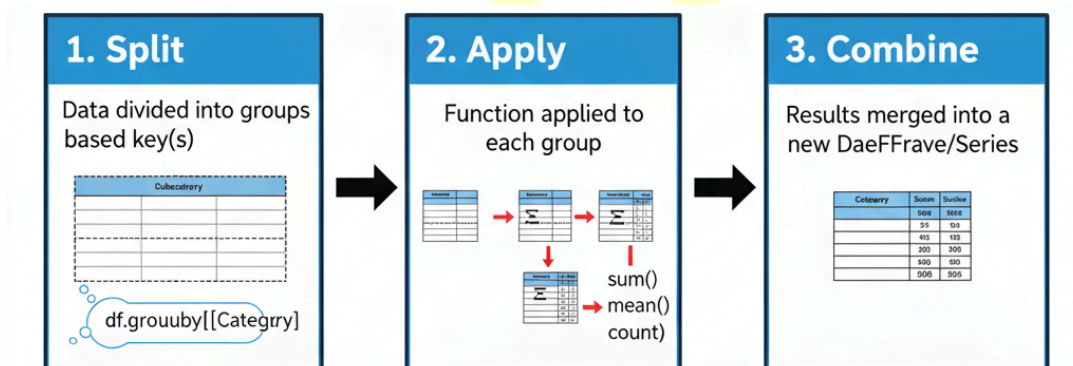
3. groupby — Aggregation by Groups

The `groupby` method is used to split the data into groups based on some key(s), apply aggregate or transformation functions, and combine results. This is often described as “split-apply-combine.”

How it works:

Split → Apply → Combine.

1. **Split:** The data is divided into groups based on one or more columns.
2. **Apply:** A function (such as `sum`, `mean`, `count`, `custom`) is applied to each group.
3. **Combine:** The results are combined into a new DataFrame or Series.

**Basic usage:**

```
df.groupby('Category')['Value'].sum()
```

This groups rows by `Category`, and for each group computes the sum of the `Value` column.

Multiple columns:

```
df.groupby(['Category', 'Subcategory'])['Value'].mean()
```

Important parameters / methods:

- `as_index=False`: prevents grouping columns from becoming the index in the result. (Keeps them as normal columns)

- `agg()`: apply multiple aggregation functions or different aggregation per column, by passing a dictionary.

Example:

```
df.groupby('Category').agg({'Value': ['sum', 'mean'], 'Other': 'max'})
```

- `filter()`: used to drop entire groups based on a condition (for example, keep only groups whose size > some threshold).
- `transform()`: produce an output that has the same shape as the original, but transformed values per group.
- `get_group(group_key)`: retrieves the subset corresponding to one group.
- `groups`: attribute that maps group keys to the indices (rows) in each group.

GroupBy objects allow flexible summarization. They work like SQL's `GROUP BY` but are fully integrated with Python. When you iterate over a GroupBy object:

```
for key, subdf in df.groupby('Category'):
    print(key)
    print(subdf.head())
```

This shows each group's key and corresponding sub-DataFrame.

Real-world Example:

A store can group sales by Region and Month to find average revenue per region — enabling comparative analysis.

4. merge / join / concat — Combining DataFrames

Often data is spread across multiple DataFrames (tables) and we need to combine them.

`merge()`

This is analogous to SQL joins. It allows you to combine two DataFrames based on one or more keys (columns).

Syntax:

```
pd.merge(left_df, right_df, how='inner', on='key')
```

- **Types of joins:**
 - `inner`: only keys present in both (intersection)
 - `left`: all keys from left, matching from right (right missing => NaN)
 - `right`: all keys from right
 - `outer`: union of keys, missing values filled with NaN

It can also specify `left_on` / `right_on` if key columns have different names.

`concat()`

Used to stack DataFrames either **vertically** (one below another) or **horizontally** (side by side). It's simpler than merge when we just want to append rows or columns.

```
pd.concat([df1, df2], axis=0) # Add rows
pd.concat([df1, df2], axis=1) # Add columns
```

Parameters like `join='inner'` or `'outer'` and `ignore_index=True` control how indices are treated.

5. `pivot_table` and Reshaping

Pivot tables are used to create **summarized, multi-dimensional views** of data by distributing aggregation over rows and columns. It is a reshaping tool.

Basic syntax:

```
pd.pivot_table(
    data=df,
    values='ValueColumn',
    index=['RowKey1', ...],
    columns=['ColumnKey1', ...],
    aggfunc='sum' or 'mean' or custom,
    fill_value=0,
    margins=False
)
```

- `index`: rows by which grouping is done (these become the new index)
- `columns`: columns that become new columns in the pivot
- `values`: the column whose values you aggregate
- `aggfunc`: aggregation function, e.g., `sum`, `mean`, `count`
- `fill_value`: value to use for missing cells
- `margins=True`: adds totals (row and column)

Example:

If we have sales data with columns `'Product'`, `'Region'`, `'Sales'`, we could make a pivot table that shows total sales of each product in each region:

```
pv = pd.pivot_table(df,
                    values='Sales',
                    index='Product',
                    columns='Region',
                    aggfunc='sum',
                    fill_value=0)
```

This gives a table where rows are Products, columns are Regions, and the cells show the summed sales. Pivot tables are conceptually similar to `groupby` + `unstack`, but more convenient for multi-dimensional summarization.

6. Example: Combining Everything

A realistic workflow demonstrating all key concepts (reading, cleaning, grouping, merging, and pivoting):

```
import pandas as pd
```

```
# 1. Read data
```

```
df1 = pd.read_csv('sales.csv')      # columns: OrderID, Product, Amount, Region
```

```
df2 = pd.read_csv('products.csv')   # columns: Product, Category
```

```
# 2. Clean data
```

```
df1 = df1.dropna(subset=['Amount'])  
df1['Amount'] = df1['Amount'].astype(float)
```

```
# 3. Merge product category into sales
```

```
df = pd.merge(df1, df2, how='left', on='Product')
```

```
# 4. Group by Category, Region and compute totals
```

```
summary = df.groupby(['Category', 'Region']).agg({'Amount':  
'sum'}).reset_index()
```

```
# 5. Pivot the summary
```

```
pivot = pd.pivot_table(summary,  
                          values='Amount',  
                          index='Category',  
                          columns='Region',  
                          aggfunc='sum',  
                          fill_value=0)
```

```
print(pivot)
```

This will output a table where each row is a *Category*, each column is a *Region*, and each cell is total sales.

Explanation:

- Step 1: Load data
 - Step 2: Clean missing and incorrect entries
 - Step 3: Merge two datasets
 - Step 4: Group by key attributes for analysis
 - Step 5: Reshape using a pivot table for reporting
-

Function	Purpose	Analogy
Series	1D labeled array	Single column
DataFrame	2D labeled data	Excel sheet
groupby()	Split-Apply-Combine aggregation	SQL GROUP BY
merge()	Combine DataFrames on keys	SQL JOIN
pivot_table()	Multi-dimensional summarization	Excel Pivot

Practical Exercise

Level 1: Basic

Exercise: DataFrame Creation / Basic operations

```
# Task: You need to:
# 1. Create a DataFrame with student data.
# 2. 3 columns: Name, Marks, Grade.
# 3. Add at least 5 rows.
# 4. Filter students whose marks are greater than 70.

# Try the solution:
import pandas as pd
data = {
    'Name': ['Ali', 'Sana', 'Ahmed', 'Fatima', 'Bilal'],
    'Marks': [85, 92, 65, 78, 88],
    'Grade': ['A', 'A', 'C', 'B', 'A']
}
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)

# Filter high marks students
high_marks = df[df['Marks'] > 70]
print("\nHigh Marks Students:")
print(high_marks)
```

Level 2: GroupBy

Exercise: Sales Data Analysis

```
# Task: Analyze the company's sales data
# 1. Group the sales data by 'Product Category'
# 2. Calculate the total sales for each category
# 3. Also, find the average sales
```



```
# Data:
sales_data = {
    'Category': ['Electronics', 'Clothing', 'Electronics', 'Food',
'Clothing'],
    'Product': ['Laptop', 'Shirt', 'Phone', 'Rice', 'Pants'],
    'Sales': [50000, 3000, 25000, 2000, 4000]
}
df_sales = pd.DataFrame(sales_data)

# Write your solution here:
# Group by category
category_group = df_sales.groupby('Category')['Sales']

# Total sales per category
total_sales = category_group.sum()
print("Total Sales per Category:")
print(total_sales)

# Average sales per category
avg_sales = category_group.mean()
print("\nAverage Sales per Category:")
print(avg_sales)
```

Level 3: Merge/Pivot Table

Exercise: Multiple Datasets combining

Task: Merge two different data sets and create a pivot table

```
# Dataset 1: Employee Basic Info
employees = {
    'EmpID': [101, 102, 103, 104],
    'Name': ['Ali', 'Sana', 'Ahmed', 'Fatima'],
    'Department': ['IT', 'HR', 'IT', 'Finance']
}
```

```
# Dataset 2: Employee Salary Info
salaries = {
    'EmpID': [101, 102, 103, 104],
    'Salary': [50000, 45000, 55000, 48000],
    'Bonus': [5000, 4000, 6000, 4500]
}
```

```
# You need to:
# 1. Merge both datasets
# 2. Calculate the average salary by department
# 3. Create a pivot table for a department-wise summary
```

Try your code here:


```
df_emp = pd.DataFrame(employees)
df_sal = pd.DataFrame(salaries)

# Merge datasets
merged_df = pd.merge(df_emp, df_sal, on='EmpID')
print("Merged DataFrame:")
print(merged_df)

# Department-wise analysis
dept_analysis = merged_df.groupby('Department').agg({
    'Salary': 'mean',
    'Bonus': 'mean'
})
print("\nDepartment-wise Analysis:")
print(dept_analysis)
```

Common Panda Error & Quick Fixes

1. KeyError (Column Not Found)

```
# ERROR
df['column_name']

# FIXES
print(df.columns.tolist())      # Check available columns
df.get('column_name')           # Safe method - returns None if missing
```

2. SettingWithCopyWarning

```
# ERROR
filtered_df = df[df['age'] > 30]
filtered_df['new_col'] = 'value'  # Warning!

# FIX
filtered_df = df[df['age'] > 30].copy()
filtered_df['new_col'] = 'value'
```

3. NaN Values Issues

```
# Check NaN
print(df.isnull().sum())

# Fix NaN
df['col'].fillna(0, inplace=True)      # Fill with 0
df['col'].fillna(df['col'].mean(), inplace=True)  # Fill with mean
df.dropna(subset=['col'], inplace=True)  # Remove rows with NaN
```

4. Data Type Problems

```
# String to number
df['price'] = df['price'].str.replace(',', '').astype(float)
df['price'] = pd.to_numeric(df['price'], errors='coerce') # Safe conversion
```

5. Merge Column Conflicts

```
# Duplicate columns after merge
merged = pd.merge(df1, df2, on='id', suffixes=('_left', '_right'))
```

6. GroupBy Confusion

```
# GroupBy returns object, not DataFrame
result = df.groupby('category')['value'].mean().reset_index()
```

7. Slow Performance

```
# Use vectorization instead of loops
df['new'] = df['col1'] + df['col2'] # FAST

# Optimize data types
df['category'] = df['category'].astype('category')
```

8. DateTime Issues

```
df['date'] = pd.to_datetime(df['date_string'], errors='coerce')
```

Quick Debugging Checklist

```
print(df.shape)           # Check dimensions
print(df.columns)         # Check column names
print(df.dtypes)          # Check data types
print(df.isnull().sum())  # Check missing values
```
